

Tutorial - Semana 5 - Encontro 2

Introdução

No Encontro 1, o Vertical Slice ganhou um contrato de comunicação: `BPI_Interactive`, implementada em um Actor de teste e já chamada de forma genérica pelo `InteractionComponent` de `BP_Player`. Faltava resolver o problema complementar: como o objeto ativado avisa sua própria reação a quem quiser ouvir, sem precisar conhecer quem está ouvindo. Este encontro constrói um Event Dispatcher para isso e fecha a semana aplicando os dois recursos — Interface + Event Dispatcher — a um objeto interativo concreto do Vertical Slice (porta ou equivalente escolhido pelo grupo), com o primeiro Feedback Formal da disciplina.

Este tutorial não substitui a explicação do professor em sala. Ele existe para que você possa acompanhar a implementação passo a passo durante o laboratório e revisar os passos depois da aula, sem depender da documentação oficial da Epic.

Objetivos da Semana

- Explicar Event Dispatchers como padrão observer para notificação de eventos entre sistemas.
- Comparar Event Dispatchers com UnityEvent/C# Actions.
- Implementar um Event Dispatcher acionado por interação e aplicá-lo a um objeto interativo concreto do Vertical Slice.

Resultado Esperado ao Final da Semana

Um objeto interativo funcional integrado ao Vertical Slice: um Actor implementando `BPI_Interactive`, disparando um Event Dispatcher próprio ao ser ativado, e reagindo de forma visível a uma ação do jogador via `InteractionComponent`, com mecanismo de acionamento definido pelo grupo.

Pré-requisitos

- `BPI_Interactive` do Encontro 1 implementada em ao menos um Actor do nível de teste.
 - `InteractionComponent` de `BP_Player` já chamando `Interact` via "Call Function on Interface".
-

Antes de começar

O que você deverá possuir antes desta semana

- O projeto do Encontro 1, com `BPI_Interactive` implementada e a chamada via interface funcionando no Actor de teste.

Arquivos necessários

- Nenhum arquivo externo é necessário neste encontro.

Assets utilizados

- Nenhum asset novo obrigatório. Caso o grupo escolha uma porta ou alavanca como objeto interativo, pode reutilizar meshes já disponíveis no projeto ou primitivas simples (cubo, cilindro) para representar o mecanismo.

Projeto esperado

O projeto do Encontro 1, com o Actor candidato a objeto interativo (porta, alavanca ou equivalente) identificado no nível de teste.

Parte 1 — Event Dispatcher: avisando sem conhecer quem escuta

Objetivo

Compreender Event Dispatchers como padrão observer e criar um Event Dispatcher acionado pela função `Interact` da interface.

Conceito

Se a interface resolve "quem pode ser chamado sem conhecer a classe concreta", o Event Dispatcher resolve o problema inverso: "quem precisa ser avisado quando algo acontece, sem que o emissor do evento precise conhecer os interessados". É o **padrão observer**: um objeto declara um evento (por exemplo, "fui ativado"), e qualquer outro sistema pode se inscrever para reagir, sem que o objeto que dispara o evento saiba quantos ou quais sistemas estão ouvindo. Isso é o que permite, por exemplo, que uma porta avise sua própria animação de abertura e, ao mesmo tempo, um sistema de áudio ou de missão no futuro da disciplina, sem que a porta tenha uma lista fixa de "quem reagir".

Passo a passo

1. Abrir o Actor que já implementa `BPI_Interactive` (do Encontro 1) ou o Actor escolhido pelo grupo para ser o objeto interativo definitivo desta semana (porta, alavanca ou equivalente).
2. No painel Variables desse Actor, na seção inferior, criar um novo Event Dispatcher (botão "+" ao lado de "Event Dispatchers").
3. Nomear o Event Dispatcher de forma descritiva (por exemplo, `OnAtivado` ou `OnInteragido`), evitando nomes genéricos como `Event1`.
4. Dentro da implementação da função `Interact` (herdada de `BPI_Interactive`), adicionar o nó "Call" do Event Dispatcher criado, disparando-o no momento em que a interação ocorre.
5. Compilar e salvar o Actor.

Resultado esperado

Um Event Dispatcher criado no Actor interativo, disparado dentro da implementação de `Interact`, pronto para ser conectado a uma ou mais reações.

Verificando

Adicionar temporariamente um Print String conectado diretamente ao evento correspondente ao Dispatcher (via "Bind Event to..." no próprio Event Graph do Actor, ou testando o disparo no Event Graph) e confirmar, em modo Play, que o Print String aparece ao acionar a interação.

Problemas comuns

- **Confundir Event Dispatcher com uma função comum:** Event Dispatchers não retornam valor e podem ter múltiplos inscritos; se a necessidade é apenas uma chamada direta e única, uma função resolve — mas o padrão observer exige o Dispatcher.

- **Disparar o Dispatcher fora da função `Interact`** : o disparo deve ocorrer dentro da lógica que a interface já garante ser chamada de forma desacoplada; disparar em outro ponto do Event Graph quebra a ligação entre interação e notificação.
- **Nomear o Dispatcher de forma genérica:** nomes como `Event1` dificultam a leitura do Blueprint por outros membros do grupo; sempre nomear pelo que o evento representa (`OnAtivado` , `OnPortaAberta`).

Boas práticas

Mantenha o Event Dispatcher disparando apenas a notificação ("fui ativado"), sem carregar lógica de reação dentro de si — a reação pertence a quem se inscreve (Bind Event), não ao próprio disparo.

Comparação com Unity

UnityEvent e C# Actions (`Action` , `Action<T>`) resolvem o mesmo problema de notificação sem acoplamento direto: um objeto expõe um evento, outros se inscrevem via `AddListener` ou `+=` , e o emissor nunca precisa conhecer os inscritos. A diferença arquitetural relevante é de camada — Event Dispatchers são configuráveis visualmente no Blueprint Editor e podem ser conectados sem escrever código, enquanto UnityEvent exige exposição via Inspector ou código, e C# Actions exigem código puro.

Parte 2 — Aplicando Interface + Event Dispatcher a um objeto interativo concreto

Objetivo

Combinar `BPI_Interactable` e o Event Dispatcher em um objeto interativo concreto do Vertical Slice, com mecanismo de acionamento definido pelo grupo.

Conceito

Um objeto interativo completo do Vertical Slice não é apenas uma função chamada via interface — é a combinação de duas peças complementares: o contrato que permite ser chamado (`BPI_Interactable`) e a notificação que avisa sua própria reação (Event Dispatcher). O mecanismo de acionamento (alavanca, chave, proximidade) é uma decisão de design do grupo, mas a

estrutura de comunicação — interface para ser chamado, Dispatcher para notificar — é a mesma para qualquer objeto interativo que o Vertical Slice acumular nas próximas semanas (baús, checkpoints, pickups).

Passo a passo

1. Definir, em grupo, o objeto interativo (porta, alavanca ou equivalente) e o mecanismo de acionamento (alavanca separada, chave coletável simulada, proximidade ou outro).
2. Garantir que o objeto interativo final implementa `BPI_Interactable` e possui o Event Dispatcher criado na Parte 1.
3. Dentro do próprio Actor, no Event Graph, usar "Bind Event to [NomeDoDispatcher]" (ou conectar diretamente ao evento correspondente) para inscrever a reação definitiva — por exemplo, rotação ou translação da porta, troca de estado visual, ou ativação de outro Actor relacionado ao mecanismo escolhido.
4. Caso o mecanismo de acionamento envolva um segundo Actor (por exemplo, uma alavanca separada que abre uma porta), garantir que esse segundo Actor também implementa `BPI_Interactable` e, ao ser ativado, dispare o Event Dispatcher do objeto principal (via referência direta ou outro mecanismo definido pelo grupo).
5. Compilar e salvar todos os Blueprints envolvidos.
6. Testar em modo Play: acionar o objeto interativo pelo mecanismo escolhido e confirmar que a reação definitiva ocorre de forma visível, disparada pelo Event Dispatcher.
7. Preparar a apresentação do mecanismo de acionamento escolhido para o Feedback Formal ao final do encontro.

Resultado esperado

Um objeto interativo funcional no nível de teste, implementando `BPI_Interactable`, disparando um Event Dispatcher próprio ao ser ativado, e reagindo de forma visível e demonstrável à ação do jogador via `InteractionComponent`, com mecanismo de acionamento definido pelo grupo.

Verificando

Acionar o mecanismo escolhido em modo Play e confirmar que a reação definitiva do objeto interativo ocorre de forma visível e consistente, sem depender de nós de teste temporários (como o Print String da Parte 1, que deve ser removido ou desativado nesta etapa).

Problemas comuns

- **Chamar a reação diretamente da função `Interact` , sem passar pelo Event Dispatcher:** isso resolve o desafio "por fora" do padrão ensinado; pergunte "e se outro sistema também precisasse reagir a essa ativação?" — se a resposta for sim, a reação deve passar pelo Dispatcher.
- **Deixar nós de teste temporários (`Print String`) no Blueprint final:** remover ou desativar esses nós antes da apresentação do Feedback Formal.
- **Mecanismo de acionamento com dois Actors sem referência clara entre eles:** garantir que o Actor que ativa o mecanismo (alavanca) tenha uma forma explícita de referenciar ou notificar o Actor principal (porta), sem duplicar a lógica de interação.

Boas práticas

Ao escolher o mecanismo de acionamento, prefira soluções que reutilizem `BPI_Interactable` também no segundo Actor (quando houver), mantendo a mesma estrutura de comunicação desacoplada em todo o sistema, em vez de criar um caminho especial só para esse caso.

Comparação com Unity

O equivalente, em Unity, seria o objeto expor um `UnityEvent` (ou `Action`) público, e outros scripts se inscreverem via `AddListener` no `Start()` ou `OnEnable()` . O princípio — observer, inversão de controle da notificação — é o mesmo nas duas engines; a diferença está na camada de configuração, visual na Unreal e majoritariamente por código na Unity.

Desafio

Cada grupo implementa um objeto interativo (porta ou equivalente escolhido pelo grupo) usando `BPI_Interactable` + Event Dispatcher, com liberdade sobre o mecanismo de acionamento — alavanca, chave, proximidade ou outra solução própria —, desde que o objeto reaja de forma demonstrável a uma ação do jogador via `InteractionComponent` .

Ao final da semana

Ao final da Semana 5 (Encontros 1 e 2), o Vertical Slice deve possuir um objeto interativo funcional integrado: um Actor implementando `BPI_Interactable` , disparando um Event Dispatcher próprio ao ser ativado, e reagindo de forma visível a uma ação do jogador via `InteractionComponent` ,

com mecanismo de acionamento definido pelo grupo. Isso corresponde ao início da linha "Interactables" do roadmap descrito no PROJECT_ARCHITECTURE.md (seção 7) — `BPI_Interactable` e os Event Dispatchers de interação passam a ser a base de comunicação reutilizada por todo objeto interativo futuro (`BP_Chest` , `BP_Pickup` , `BP_Checkpoint`).

Checklist

- ☐ Event Dispatcher criado no Actor interativo, disparado dentro da implementação de `Interact`
- ☐ Nome do Event Dispatcher descritivo, não genérico
- ☐ Reação definitiva inscrita via Bind Event, não chamada diretamente de `Interact`
- ☐ Objeto interativo funcional com mecanismo de acionamento definido pelo grupo
- ☐ Nós de teste temporários removidos do Blueprint final
- ☐ Feedback Formal registrado ao final do encontro

Glossário

- **Event Dispatcher:** recurso da Unreal que permite a um objeto declarar um evento e a outros sistemas se inscreverem para reagir a ele, sem que o emissor conheça os inscritos.
- **Bind Event to:** nó de Blueprint usado para inscrever uma função ou sequência de nós a um Event Dispatcher, executada sempre que o Dispatcher é chamado.
- **Padrão observer:** padrão de arquitetura de software em que um objeto notifica múltiplos interessados sobre um evento sem depender diretamente deles.

Referências

- EPIC GAMES. **Unreal Engine 5 Documentation** — Blueprints Visual Scripting in Unreal Engine. Disponível em: <https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprints-visual-scripting-in-unreal-engine>.
- EPIC GAMES. **Unreal Engine Learning Library** — introdução a Event Dispatchers. Disponível em: <https://dev.epicgames.com/community/unreal-engine/learning>.
- UNITY TECHNOLOGIES. **Unity Manual** — UnityEvent, para fins comparativos. Disponível em: <https://docs.unity3d.com/Manual/>.
- Vídeo sugerido (apoio complementar, nunca substituindo a documentação oficial): canal oficial **Unreal Engine**, vídeos introdutórios de Event Dispatchers; **Mathew Wadstein**, explicação pontual de WTF Is? Event Dispatcher.